

Machete de Comandos | Clase 3

GitHub, Codespaces y control de versiones · Seminario de Actualización · IFTS N.º 18

Referencia rápida de todo lo usado en la Clase 3. Guardalo — lo vas a necesitar para la actividad de la semana.

Repaso: entorno local

Comando	Qué hace
<code>python --version</code>	Ver la versión de Python instalada
<code>python -m venv .env</code>	Crear un entorno virtual
<code>.env\Scripts\activate</code>	Activar el entorno virtual (Windows)
<code>source .env/bin/activate</code>	Activar el entorno virtual (Mac/Linux/Codespaces)
<code>pip install -r requirements.txt</code>	Instalar las dependencias del proyecto
<code>git add .</code>	Preparar los cambios para el próximo commit
<code>git commit -m "mensaje"</code>	Guardar los cambios preparados en el historial

GitHub Codespaces

No son comandos de terminal — son pasos en la interfaz web de GitHub:

1. En el repo de GitHub, botón verde "Code".
2. Pestaña "Codespaces".
3. "Create codespace on main".

Recordar: por dentro corre Linux (no Windows) — usá siempre `source .env/bin/activate` ahí. El plan gratuito tiene un límite de horas por mes (hoy, 60 hs con cuenta personal).

Copilot: prompts de ejemplo

- "¿Cómo creo y activo un entorno virtual de Python en esta terminal?"
- "Este comando me tira este error: [pegar error]. ¿Qué significa y cómo lo soluciono?"
- "Generame un requirements.txt con gradio y pandas."
- "Generame un .gitignore para un proyecto Python con entorno virtual y Gradio."

Conectar el repo local con GitHub

Comando	Qué hace
<code>git remote -v</code>	Ver los remotos conectados al repo
<code>git remote add origin <url></code>	Conectar el repo local con uno en GitHub
<code>git push -u origin main</code>	Subir los commits la primera vez; asocia tu rama local "main" con la del remoto
<code>git push</code>	Subir commits nuevos (después del primer push -u)

Comando	Qué hace
git clone <url>	Clonar un repositorio existente en tu compu

origin es un alias por convención (no una palabra especial de Git). En git push -u origin main, "main" es el nombre de tu rama LOCAL, la que se asocia con la del mismo nombre en el remoto.

Ramas

Comando	Qué hace
git branch	Listar las ramas locales; * marca la activa
git checkout nombre_rama	Cambiar de rama
git switch nombre_rama	Cambiar de rama (forma más nueva)
git branch -m nombre_viejo nombre_nuevo	Renombrar una rama
git push origin --delete nombre_rama	Borrar una rama del remoto
git config --global init.defaultBranch main	Configurar "main" como nombre por defecto para ramas nuevas

Otros comandos útiles

Comando	Qué hace
git rm -r --cached carpeta	Dejar de trackear una carpeta sin borrarla de la compu
git pull origin main --allow-unrelated-histories	Traer y fusionar un remoto con commits que el local no tiene
git config --global user.name "Tu Nombre"	Configurar el nombre para tus commits
git config --global user.email "vos@mail.com"	Configurar el mail para tus commits
git log --oneline --graph	Ver el historial de commits en una sola vista

Solución rápida a los 3 problemas de hoy

1 • Ya tenía README y no me deja hacer push

```
git pull origin main --allow-unrelated-histories
# resolver el conflicto si los dos README chocan
git push
```

2 • Subí una carpeta que no debía

```
git rm -r --cached nombre_carpeta
git commit -m "dejar de trackear nombre_carpeta"
git push
# recién ahora agregarla al .gitignore
```

3 • Me quedaron dos ramas, master y main

```
git branch -m master main  
git push -u origin main  
git push origin --delete master  
git config --global init.defaultBranch main
```